



SprintCode.pro

Подготовка к алгоритмическим задачам

[Главная](#) / [Блог](#) / [Что такое жадный алгоритм: простое объяснение с примерами](#)

11 мин чтения

[алгоритмы](#)

[жадные алгоритмы](#)

[оптимизация](#)

[сортировка](#)

[локальный оптимум](#)

Почему жадные алгоритмы важны?

Жадные алгоритмы — это фундаментальный подход к решению оптимизационных задач, который позволяет:

- 🧠 Получать быстрые решения сложных оптимизационных проблем
- ⚡ Существенно сокращать время работы по сравнению с полным перебором
- 🎯 Находить точные решения для определённого класса задач
- 🔍 Получать приближённые ответы с гарантированной точностью
- ⭐ Разрабатывать интуитивно понятные и легко реализуемые алгоритмы

Принцип работы жадных алгоритмов

Основная идея:

Жадный алгоритм строит решение пошагово, выбирая на каждом шаге локально оптимальное решение (максимально выгодное в текущий момент), не пересматривая сделанных ранее выборов. В отличие от динамического программирования или полного перебора, жадный алгоритм никогда не отменяет принятых решений.

Ключевые компоненты:

1. **Функция выбора:** Определяет, какой элемент выбрать на текущем шаге
2. **Критерий осуществимости:** Проверяет, можно ли добавить элемент к текущему решению



Когда работают жадные алгоритмы:

Жадные алгоритмы гарантированно находят оптимальное решение, если задача обладает двумя свойствами:

1. **Свойство жадного выбора:** Локально оптимальный выбор приводит к глобально оптимальному решению
2. **Свойство оптимальной подструктуры:** Оптимальное решение задачи содержит оптимальные решения подзадач

Пример 1: Задача о выборе активностей

Формулировка задачи:

Дан набор из n активностей с временем начала и окончания: $[start_1, end_1]$, $[start_2, end_2]$, ..., $[start_n, end_n]$. Необходимо выбрать максимальное количество не пересекающихся по времени активностей.





Платформа для подготовки

Решай алгоритмические задачи как профи

✓ Популярные алгоритмы

✓ Разбор решений

✓ AI помощь

Начать сейчас →

Пример входных данных:

Активности:

```
1: [1, 4]   # Начало: 1, Окончание: 4
2: [3, 5]   # Начало: 3, Окончание: 5
3: [0, 6]   # Начало: 0, Окончание: 6
4: [5, 7]   # Начало: 5, Окончание: 7
5: [3, 9]   # Начало: 3, Окончание: 9
6: [5, 9]   # Начало: 5, Окончание: 9
7: [6, 10]  # Начало: 6, Окончание: 10
8: [8, 11]  # Начало: 8, Окончание: 11
9: [8, 12]  # Начало: 8, Окончание: 12
10: [2, 14] # Начало: 2, Окончание: 14
```

Жадный подход к решению:

1. Отсортировать активности по времени окончания
2. Выбрать первую активность
3. Последовательно выбирать каждую следующую активность, которая не пересекается с уже выбранными

```
function selectActivities(activities) {
  // Сортируем активности по времени окончания
  activities.sort((a, b) => a[1] - b[1])

  const selected = [activities[0]] // Выбираем первую активность
  let lastEndTime = activities[0][1] // Время окончания последней выбранной акти
```



```
// Если текущая активность начинается после окончания последней выбранной
if (start >= lastEndTime) {
    selected.push(activities[i])
    lastEndTime = end
}
}

return selected
}

// Пример использования
const activities = [
    [1, 4],
    [3, 5],
    [0, 6],
    [5, 7],
    [3, 9],
    [5, 9],
    [6, 10],
    [8, 11],
    [8, 12],
    [2, 14],
]

const selectedActivities = selectActivities(activities)
console.log(selectedActivities)
// Вывод: [[1, 4], [5, 7], [8, 11]]
```

Почему жадный алгоритм здесь работает?

1. **Свойство жадного выбора:** Выбор активности с самым ранним временем окончания всегда позволяет оставить максимальное время для оставшихся активностей
2. **Оптимальная подструктура:** После выбора первой активности, задача сводится к выбору максимального числа непересекающихся активностей из оставшихся

Временная сложность: $O(n \log n)$ из-за сортировки

Пространственная сложность: $O(n)$ для хранения отсортированного массива и результата

Пример 2: Задача о размене монет

Формулировка задачи:



Пример входных данных:

Номиналы монет: [1, 5, 10, 25] (центы)

Сумма: 63 цента

Жадный подход к решению:

1. Отсортировать номиналы монет по убыванию
2. Последовательно брать максимально возможное количество монет каждого номинала

```
function coinChange(coins, amount) {  
    // Сортируем номиналы по убыванию  
    coins.sort((a, b) => b - a)  
  
    let remainingAmount = amount  
    let coinCount = 0  
    const usedCoins = {}  
  
    for (const coin of coins) {  
        // Количество монет текущего номинала  
        const count = Math.floor(remainingAmount / coin)  
  
        if (count > 0) {  
            usedCoins[coin] = count  
            coinCount += count  
            remainingAmount -= coin * count  
        }  
    }  
  
    // Проверяем, удалось ли разменять всю сумму  
    if (remainingAmount === 0) {  
        return { coinCount, usedCoins }  
    } else {  
        return 'Невозможно разменять сумму данными номиналами'  
    }  
}  
  
// Пример использования  
const coins = [1, 5, 10, 25]  
const amount = 63  
  
const result = coinChange(coins, amount)
```



Особенности этой задачи:

1. **Частный случай:** Жадный алгоритм дает оптимальное решение для стандартных номиналов монет США [1, 5, 10, 25], но не гарантирует оптимальность для произвольного набора номиналов
2. **Контрпример:** Для номиналов [1, 3, 4] и суммы 6, жадный алгоритм дает решение из 3 монет (4 + 1 + 1), хотя оптимальное решение — 2 монеты (3 + 3)

Временная сложность: $O(n \log n + \text{amount}/\text{minCoin})$ в худшем случае

Пространственная сложность: $O(n)$

Пример 3: Дробная задача о рюкзаке

Формулировка задачи:

Дан набор предметов, каждый с весом и ценностью. Необходимо заполнить рюкзак ограниченной вместимости предметами так, чтобы их суммарная ценность была максимальной. В отличие от 0/1 задачи о рюкзаке, здесь можно брать дробные части предметов.

Пример входных данных:

Предметы: [(10, 60), (20, 100), (30, 120)] (вес, ценность)

Вместимость рюкзака: 50

Жадный подход к решению:

1. Вычислить удельную ценность (ценность/вес) для каждого предмета
2. Отсортировать предметы по удельной ценности по убыванию
3. Последовательно добавлять предметы в рюкзак, начиная с наиболее ценных за единицу веса

```
function fractionalKnapsack(items, capacity) {  
  // Вычисляем удельную ценность для каждого предмета  
  const valuePerWeight = items.map((item, index) => ({  
    index,  
    weight: item[0],  
    value: item[1],  
    ratio: item[1] / item[0],  
  })))
```



```
let totalValue = 0
let remainingCapacity = capacity
const selectedItems = []

for (const item of valuePerWeight) {
  // Если можем взять весь предмет
  if (item.weight <= remainingCapacity) {
    selectedItems.push({
      index: item.index,
      weight: item.weight,
      value: item.value,
      fraction: 1,
    })

    totalValue += item.value
    remainingCapacity -= item.weight
  } else {
    // Берем только часть предмета
    const fraction = remainingCapacity / item.weight

    selectedItems.push({
      index: item.index,
      weight: item.weight * fraction,
      value: item.value * fraction,
      fraction,
    })

    totalValue += item.value * fraction
    break // Рюкзак заполнен
  }
}

return { totalValue, selectedItems }
}

// Пример использования
const items = [
  [10, 60],
  [20, 100],
  [30, 120],
]
const capacity = 50

const result = fractionalKnapsack(items, capacity)
```



Почему жадный алгоритм здесь работает?

1. **Свойство жадного выбора:** Взятие предмета с максимальной удельной ценностью на каждом шаге приводит к оптимальному решению
2. **Возможность дробления:** Ключевое отличие от 0/1 задачи о рюкзаке, где жадный алгоритм не оптимален

Временная сложность: $O(n \log n)$ из-за сортировки

Пространственная сложность: $O(n)$

Пример 4: Алгоритм Хаффмана для сжатия данных

Формулировка задачи:

Дан набор символов и их частоты. Необходимо построить префиксный код с минимальной средней длиной.

Жадный подход к решению:

1. Создать очередь с приоритетами, содержащую все символы и их частоты
2. Последовательно извлекать два узла с наименьшими частотами и объединять их в новый узел
3. Добавлять новый узел обратно в очередь
4. Повторять, пока в очереди не останется один узел — корень дерева Хаффмана

```
class Node {
  constructor(char, freq, left = null, right = null) {
    this.char = char
    this.freq = freq
    this.left = left
    this.right = right
    this.code = ''
  }
}

function buildHuffmanTree(text) {
  // Подсчитываем частоты символов
  const frequencies = {}
  for (const char of text) {
    frequencies[char] = (frequencies[char] || 0) + 1
  }
}
```



```
.map(([char, freq]) => new Node(char, freq))
.sort((a, b) => a.freq - b.freq)

// Строим дерево Хаффмана
while (priorityQueue.length > 1) {
  // Извлекаем два узла с наименьшими частотами
  const left = priorityQueue.shift()
  const right = priorityQueue.shift()

  // Создаем новый внутренний узел
  const newNode = new Node(null, left.freq + right.freq, left, right)

  // Вставляем новый узел в очередь с сохранением порядка
  let i = 0
  while (i < priorityQueue.length && priorityQueue[i].freq < newNode.freq) {
    i++
  }
  priorityQueue.splice(i, 0, newNode)
}

// Корень дерева Хаффмана
return priorityQueue[0]
}

// Генерация кодов для каждого символа
function generateCodes(node, prefix = '') {
  if (node) {
    // Если это лист (символьный узел)
    if (node.char) {
      node.code = prefix
      return { [node.char]: prefix }
    }

    // Рекурсивно обходим дерево
    const leftCodes = generateCodes(node.left, prefix + '0')
    const rightCodes = generateCodes(node.right, prefix + '1')

    return { ...leftCodes, ...rightCodes }
  }
  return {}
}

// Функция кодирования текста
function huffmanEncode(text) {
  const root = buildHuffmanTree(text)
  const codes = generateCodes(root)
```



```
    encodedText += codes[char]
  }

  return { encodedText, codes, root }
}

// Пример использования
const text = 'this is an example for huffman encoding'
const { encodedText, codes } = huffmanEncode(text)

console.log('Коды Хаффмана:', codes)
console.log('Закодированный текст:', encodedText)
```

Почему жадный алгоритм здесь работает?

1. **Свойство жадного выбора:** Объединение символов с наименьшими частотами минимизирует суммарную длину кода
2. **Оптимальная подструктура:** Оптимальный код для объединенных символов является частью глобально оптимального решения

Временная сложность: $O(n \log n)$ для построения дерева Хаффмана

Пространственная сложность: $O(n)$

Пример 5: Задача составления расписания для минимизации среднего времени завершения

Формулировка задачи:

Даны n задач с временем выполнения. Необходимо определить порядок их выполнения, минимизирующий среднее время завершения.

Жадный подход к решению:

Выполнять задачи в порядке возрастания времени их выполнения (Кратчайшая работа первой — SJF)

```
function minimizeCompletionTime(tasks) {
  // Сортируем задачи по времени выполнения
  tasks.sort((a, b) => a - b)

  let currentTime = 0
  let totalCompletionTime = 0
```



```
totalCompletionTime += currentTime
}

return {
  schedule: tasks,
  averageCompletionTime: totalCompletionTime / tasks.length,
}
}

// Пример использования
const taskTimes = [3, 1, 4, 2, 6]
const result = minimizeCompletionTime(taskTimes)

console.log('Оптимальное расписание:', result.schedule)
console.log('Среднее время завершения:', result.averageCompletionTime)
// Вывод: Оптимальное расписание: [1, 2, 3, 4, 6]
// Среднее время завершения: 7.4
```

Почему жадный алгоритм здесь работает?

1. **Математическое доказательство:** Можно доказать, что при переупорядочивании двух соседних задач с временами t_1 и t_2 , где $t_1 > t_2$, всегда выгоднее поставить более короткую задачу раньше
2. **Принцип:** Минимизация времени выполнения наибольшего числа задач

Временная сложность: $O(n \log n)$ из-за сортировки

Пространственная сложность: $O(n)$

Когда жадные алгоритмы НЕ работают?

Жадные алгоритмы не всегда находят оптимальное решение. Классические примеры:

1. 0/1 Задача о рюкзаке

Для варианта, где нельзя брать дробные части предметов, жадный выбор не гарантирует оптимальности.

Контрпример: Предметы с весами и ценностями [(10, 60), (100, 120), (20, 100)], вместимость рюкзака 120.

- Жадный алгоритм выберет 1-й и 3-й предметы с суммарной ценностью 160
- Оптимальное решение — 2-й и 3-й предметы с суммарной ценностью 220



может не найти кратчайший путь.

3. Задача о построении минимального дерева Штейнера

Жадное соединение ближайших пар вершин не всегда даёт минимальное связующее дерево для заданного набора вершин.

Практические применения жадных алгоритмов

1. Компьютерные сети и маршрутизация

- Алгоритм Дейкстры для поиска кратчайшего пути
- Алгоритм Крускала для построения минимального остовного дерева

2. Сжатие данных

- Алгоритм Хаффмана для энтропийного кодирования
- LZ77 и другие словарные методы сжатия

3. Планирование и оптимизация ресурсов

- Алгоритмы планирования процессов в операционных системах
- Управление памятью и дисковым пространством

4. Финансовые приложения

- Алгоритмы размена монет в банкоматах
- Оптимизация портфеля ценных бумаг

5. Машинное обучение

- Жадные алгоритмы в решающих деревьях
- Алгоритмы градиентного спуска для оптимизации

Рекомендации по разработке жадных алгоритмов

1. **Формулировка жадного критерия:** Определите, по какому правилу будете выбирать элементы (например, "самый ранний конец", "наибольшая ценность на единицу веса")
2. **Доказательство корректности:** Убедитесь, что жадный выбор приводит к глобальному оптимуму (часто требует математического доказательства)



4. **Оптимизация реализации:** Используйте эффективные структуры данных (например, очередь с приоритетом)

5. **Тестирование:** Проверьте алгоритм на крайних случаях и больших наборах данных

Заключение

Жадные алгоритмы представляют собой мощный инструмент для решения оптимизационных задач, обеспечивая баланс между эффективностью и простотой реализации. Хотя они применимы не ко всем задачам, понимание принципов жадных алгоритмов и условий их корректности является фундаментальным навыком в арсенале разработчика алгоритмов.

Ключевые выводы:

-  Жадные алгоритмы принимают локально оптимальные решения на каждом шаге
-  Для корректности необходимы свойства жадного выбора и оптимальной подструктуры
-  Жадные алгоритмы обычно проще и эффективнее динамического программирования, но применимы к меньшему числу задач
-  Всегда проверяйте применимость жадного подхода, ищите контрпримеры
-  В подходящих задачах жадные алгоритмы дают идеальный баланс между простотой и эффективностью

Владение жадными алгоритмами и понимание их границ применимости — важнейший навык, позволяющий эффективно решать широкий класс оптимизационных задач в программировании и других областях. 🌟

Задачи по теме

Лучшее время для покупки и продажи акций

Легко

Скользящее окно

Найдите максимальную прибыль, которую можно получить, совершив одну сделку купли-продажи. Вы можете выбрать любой день для покупки и любой последующий день для продажи.



Контейнер с водой

Средне

Два указателя

Найдите максимальное количество воды, которое может содержать контейнер, образованный двумя столбцами из массива высот.

#Массивы

#Два курсора

[Алгоритмические контесты](#)

[Продуктовые компании](#)

[Интенсивная подготовка](#)

 30 мин

Платформа для подготовки к алгоритмическим собеседованиям

—